FactoryBench: Benchmarking Machine Understanding via Question-Answering

Yanis Merzouki
ETH Zurich
ymerzouki@student.ethz.ch

Coral Izquierdo Muniz
Forgis AG
coral.izquierdo@forgis.com

Matei Ignuta-Ciuncanu
Imperial College London
mic18@imperial.ac.uk

Jonas Petersen
Forgis AG, ETH Zurich
jonas.petersen@forgis.com

PP: I think the first paragraph of the abstract should be deleted. The abstract is not a place to discuss the state-of-the-art. The second paragraph is enough.

Abstract

Time-series models are widely used in industrial monitoring tasks such as forecasting, anomaly detection, and signal analysis. While highly effective for these objectives, they are often opaque and limited in their ability to provide structured reasoning for engineering decision-making. Large language models (LLMs), in contrast, can generate coherent, context-aware explanations and support multi-step reasoning. However, general-purpose LLMs still struggle when applied directly to dense multivariate sensor streams and machine-specific diagnostics without adaptation or external tools. A promising direction is therefore to use LLM agents augmented with specialized numerical and signal-processing tools. This raises a central question: how can we verify that these systems truly understand machine behavior rather than only generating plausible text?

We introduce **FactoryBench**, a benchmark for evaluating LLM agents on machine understanding over industrial time-series data. Our contributions are threefold. First, we propose a scalable framework for generating machine-understanding question-answering tasks, built around 80 structured question templates spanning diverse reasoning scenarios. Second, we present **FactoryWave**, a dense multivariate dataset generated from one-armed robotic systems, including both collaborative and industrial manufacturing robots. Third, we construct FactoryBench as a large-scale Q&A dataset for time-series understanding in LLM agents, grounded in FactoryWave as well as open-source robotics datasets and simulations. Together, these components provide a rigorous testbed for evaluating reasoning, causal understanding, and decision support over real and simulated industrial signals.

1 Introduction

Modern industrial systems generate large volumes of multivariate time-series data from sensors, actuators, and controllers. In robotic manufacturing settings, these signals encode joint states, torques, forces, velocities, contact events, task phases, and fault indicators. Extracting actionable knowledge from such data is central to monitoring, diagnostics, anomaly detection, and decision support.

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

This first paragraph would be much better if it would refer to a real-world example. At least one or two modern industrail systems that fall into the category described here should be referenced.

Traditional time-series models excel at narrow tasks such as prediction, classification, and anomaly detection. However, these models are often specialized and difficult to interpret. They typically output labels or numeric predictions without explicit reasoning or higher-level explanations and cannot do much more outside of the specific task they are trained on, which limits their direct utility for automated engineering decision-making.

citation needed

citation needed

Large language models, in contrast, exhibit strong general reasoning abilities and can produce structured explanations in technical contexts Wei et al. [2022], Yao et al. [2023b]. They can integrate textual information, follow logical chains, and generate interpretable outputs. Yet, when applied directly to dense numerical time series, general LLMs still underperform without adaptation. At the same time, transformer-based architectures have become increasingly effective for time-series forecasting and representation learning Vaswani et al. [2017], Zhou et al. [2021], Wu et al. [2021], Zhou et al. [2022], Nie et al. [2023], Woo et al. [2024], Das et al. [2024].

citation needed

The last sentence seems to contradict the sentence before. Is this attempting to say that specialized transformer architectures are effective in contrast to general LLMs? Then I would add a quantifier like "specialized" here.

A promising paradigm is to build LLM agents equipped with specialized tools, including time-series models and signal-processing modules Schick et al. [2023], Yao et al. [2023a], Qin et al. [2023]. These agents can combine language reasoning with domain-specific computation. Nevertheless, evaluating whether such systems truly understand machine behavior remains challenging. Existing benchmarks mostly target textual reasoning, code generation, or generic multimodal understanding Hendrycks et al. [2021], Srivastava et al. [2023], Yue et al. [2024], leaving a gap in rigorous evaluation for machine-centered, time-series reasoning.

In the previous paragraph there is the phrase " whether such systems truly understand machine behavior". This phrase is a) basically the most important concept for this paper because it is essentially the gap that we are attempting to fill and b) not really that clear. I think it would make sense to really spell out here how current models fall short in understanding and what understanding means for us exactly.

To address this gap, we propose FactoryBench, a benchmark designed to evaluate machine understanding in LLM agents. FactoryBench is grounded in FactoryWave, a dense multivariate time-series dataset collected from one-armed robotic systems, including collaborative and industrial manufacturing robots. The dataset captures rich dynamic behavior under diverse operational conditions.

We also introduce a scalable question-generation framework composed of 80 structured templates spanning a wide range of reasoning types and applicable to general machine data flows. These templates enable systematic construction of question-answering tasks that probe state understanding, intervention reasoning, counterfactual analysis, and decision-making in machine contexts. Using this framework and the FactoryWave dataset, we build FactoryBench, a large-scale Q&A dataset specifically designed to evaluate time-series understanding and engineering reasoning in LLM agents.

By bridging the gap between general language reasoning and specialized time-series modeling, FactoryBench provides a principled foundation for studying machine-centered intelligence in industrial environments.

the phrase "machine understanding" is a bit ambiguous in our context. Does it mean understanding of machines like robots or machines/agents that understand something about the world. I would try to find a different phrase here.

2 Related work

2.1 LLM agents and tool use

Recent advances in LLM reasoning include prompting and deliberative strategies (e.g., Chain-of-Thought and Tree-of-Thought) that improve multi-step performance Wei et al. [2022], Yao et al. [2023b], alongside tool-augmented paradigms such as Toolformer and ReAct that enable grounded computation through external modules Schick et al. [2023], Yao et al. [2023a], Qin et al. [2023].

Currently I think FactoryWave is not really that well described. But this will probably still come.

Table 1: Comparison of FactoryBench with existing benchmarks.

Benchmark	Machine/Robot	Multivariate TS	Size	Counterfactual	Ranking	Numerical	Free-Form	Novel Dense TS
TimeSeriesExam	×	×	∼700	×	✓	×	×	×
ChatTS	×	✓	∼134k	×	✓	✓	×	×
EngineMT-QA	✓	✓	∼110k	×	✓	✓	✓	×
TSAQA	Partial	×	∼210k	×	✓	×	×	×
Time-MQA	×	✓	∼200k	×	✓	✓	✓	×
MTBench	×	✓	?	×	✓	✓	✓	×
QuAnTS	×	✓	∼150k	×	✓	✓	✓	×
CLEVR	×	×	∼1M	×	×	✓	×	×
CLEVRER	×	×	∼305k	✓	×	✓	✓	×
FactoryBench (Ours)	✓	✓	**TBD**	✓	✓	✓	✓	✓

These ideas motivate industrial agent design where symbolic reasoning must be combined with numerical workflows.

2.2 Time-series learning for industrial signals

Time-series modeling has advanced through transformer-based architectures (Informer, Autoformer, FEDformer, PatchTST) Zhou et al. [2021], Wu et al. [2021], Zhou et al. [2022], Nie et al. [2023], strong alternative baselines such as TFT and N-BEATS Lim et al. [2021], Oreshkin et al. [2020], and critical comparisons against simpler linear models Zeng et al. [2023]. Foundation-model directions (e.g., Chronos) further explore transfer and zero/few-shot adaptation Woo et al. [2024], Das et al. [2024]. In parallel, anomaly and reliability monitoring literature includes OmniAnomaly, USAD, and Deep SVDD Su et al. [2019], Audibert et al. [2020], Ruff et al. [2018], with surveys and benchmarks highlighting persistent gaps in interpretability and root-cause actionability Chalapathy and Chawla [2019], Lavin and Ahmad [2015].

2.3 Causality and benchmark gaps

Causal frameworks provide formal tools for interventions and counterfactuals Pearl [2009], Peters et al. [2017], Rubin [1974], while temporal methods such as Granger causality and modern nonlinear causal discovery support directional reasoning in time series Granger [1969], Runge et al. [2019]. Existing broad benchmarks (MMLU, BIG-bench, MMMU) Hendrycks et al. [2021], Srivastava et al. [2023], Yue et al. [2024] and classical time-series resources Laptev et al. [2015], Dau et al. [2019], Wen et al. [2022] do not directly evaluate machine-centered reasoning over industrial telemetry. FactoryBench targets this gap by jointly testing temporal interpretation, causal reasoning, and engineering decision support.

Both tables 1 and 2 are not referenced in the text. First of all, all tables should be referenced, but second, I think the table 1 cannot be understood on its own. The words "Counterfactual", "Free-Form", "Novel Dense TS" are explained at some point in the manuscript, but when I see the table I haven't read aby if these yet.

Table 2: Four levels of machine understanding in FactoryBench.

Level	Task	Example Question	Ground Truth Source	Commercial Value
1	State	"What's the current of joint 3 now?"	Sensor data	Fleet monitoring
2	Intervention	"If force in joint 3 increases *now* to X, what happens?"	Simulation (present state)	Diagnostic intervention
3	Counterfactual	"If force had increased to X *at t=20ms*, what would have happened?"	Simulation (past state)	Root cause / Capacity planning
4	Decision	"Robot stopped with error C203A. What to do?"	Manual + sensor fusion	Expert-free recovery

3 FactoryBench framework

3.1 Four levels of machine understanding

Is this some sort of standard design of levels to test understanding or did you come up with this yourself. If you created this, then it should probably be argued a bit more why this was chosen. Maybe this happens later. It would, however, be great, if this was to some extend grounded in previous work or general principles in robotics and not be adhoc.

To systematically evaluate machine understanding, we organize question-answering tasks according to a four-tier hierarchy, each probing distinct reasoning capabilities:

Level 1: State. This level assesses the agent's ability to interpret the current state of the machine, including detection of anomalies, identification of operational modes, and recognition of sensor patterns. Questions at this level require accurate extraction and interpretation of time series features.

Level 2: Intervention. At this level, the agent must reason about the consequences of interventions or events occurring at the present timestep that might perturb the distribution of machine states. Tasks include predicting the immediate impact of control actions, diagnosing faults as they arise, and understanding causal relationships in real time.

Level 3: Counterfactual. This level evaluates the agent's ability to reason about hypothetical scenarios, such as the effect of an event or intervention at a previous timestep. Questions require the agent to simulate alternative histories and assess how outcomes would differ under counterfactual conditions, while still considering the history they know.

Level 4: Decision Making. The highest level encompasses complex decision-making tasks, where the agent must generate a sequence of actions or recommendations based on the time series and a prompt. This includes troubleshooting, optimization, and planning, requiring integration of state interpretation, causal reasoning, and goal-directed synthesis.

By structuring Q&A tasks along these four levels, FactoryBench enables rigorous and granular assessment of machine understanding, from basic state recognition to advanced decision support.

Design principle: Each level builds on previous. Failure at Level N implies failure at Level $N + 1$.

Why is this so?

3.2 Answer formats

FactoryBench uses three answer formats, chosen to balance evaluation rigor with scalability. Each format supports deterministic scoring (with the exception of free form), and has been chosen in order to make questions very easily checkable, but also hard to answer in the absence of correct reasoning.

Multi-Select MC. The model is presented with four independent statements (A–D) about the outcome of an event or intervention, and must select any subset of statements. Evaluation gives a score of 1 for exact match, 0.5 for 1 mistake and 0 otherwise (random guesser is expected to get 0).

This just seems wrong. The expected points of a random guesser are clearly positive as all awarded points are

Ranking. The model is given four time-series segments or outcomes (A–D) and must order them according to a specified criterion (e.g., severity of deviation, magnitude of a signal). The answer is a

Table 3: Open-source datasets incorporated into FactoryBench.

Dataset	Robot	Size	Frequency	Task	# Anomaly Types
AURSAD	UR3e	?	100 Hz	Screwing	4
voraus-AD	UR5	?	100/500 Hz	Pick and Place	12

permutation string (e.g., `DABC`). Evaluation uses exact-match rate and Kendall's τ rank correlation against the ground-truth ordering.

Tensor. The model must output at least one specific scalar value—such as the expected sensor reading following an intervention—with no multiple-choice scaffolding. The answer is a list of floating-point numbers (possibly one). Evaluation is done on each scalar separately, and uses mean absolute percentage error (MAPE) and a threshold-based accuracy metric (prediction within $\pm k\%$ of ground truth), with partial points given to all correct scalars given.

Free-Form. The model produces an open-ended natural language response such as a diagnosis, recommended action sequence, or causal explanation. Because no single deterministic answer exists, evaluation is performed via an LLM-as-judge voting protocol: three independent frontier LLMs each compare the model's response against the ground-truth reference and cast a verdict of **Wrong** (0), **Neutral** (0.5), or **Correct** (1). The final score is the majority vote across the three judges, with ties broken by averaging.

This invites the obvious criticism that your evaluation and hence the final score depends on the LLM judges and could therefore change with time. I don't think it is a big issue, but one could address it here. (E.g. one could say that the LLM judges actually have a very simple task and hence not much fluctuation is expected. You could support this by studying how often the current judges are vote unanimously, which I would think is essentially always.)

3.3 Time series data and causal schema (SCE)

To ensure that the dataset structure itself encodes causality, FactoryBench organizes all time-series signals into three causal groups, answering the core question: *"What was the machine told to do, and what did it actually do?"*

- **Setpoint:** The controller's command—target position, velocity, and acceleration per axis.
- **Context:** Physical conditions affecting behavior—payload, temperature, material properties. Split into *static* (episode metadata) and *dynamic* (time-series columns).
- **Effort + Feedback:** The machine's response—motor current (effort), actual position (feedback), vibration, and acoustic emission.

This structure enables a universal fault definition: under healthy operation, Effort is a lawful function of Setpoint. Faults manifest as deviations in the $f(\text{Setpoint})$ vs Effort relationship. The dataset provides the paired signals that make this comparison possible.

unclear what a lawful function is

here a function f is used that was never defined

This data is sourced from:

- **FactoryWave:** A custom dataset generated from one-arm robotic platforms executing canonical industrial tasks (e.g., pick-and-place) under varied conditions and systematically injected anomalies.
- **Open Source Datasets:** Adapted datasets such as Aursad and Vorausad, offering diverse industrial scenarios and preprocessed to conform to the unified episode structure.
- **Simulations:** Synthetic time-series data generated from simulated robotic systems to enable controlled experimentation, ablation studies, and validation across both physical and virtual domains.

table 3 is never mentioned in the text.

4 Reliable Q&A generation (key contribution)

4.1 At scale generation via extensive labelling

Scalable ground-truth generation is the central challenge of any Q&A benchmark grounded in raw sensor data. FactoryBench addresses this by coupling a structured labelling ontology with a context-free grammar (CFG)-style template system, designed by PhD-level experts in robotics. Rather than annotating individual questions by hand, each template is parameterized: concrete values are filled at generation time from the episode data and its associated labels. The context time series used to fill the variables of the question is sampled uniformly by datasource (sim vs open source vs FactoryWave), dataset (if looking at open source), experiment, difficulty, length and then placement in that order. This yields a combinatorial expansion from a small set of carefully designed templates into a large, diverse question pool.

Every referee will ask if the reliance on templates can lead to models finding shortcuts. If the template structure can be learned then the task can be learned simply by emulating those and no understanding is achieved. One needs to address this issue here.

Variable sampling. Each question template contains multiple variable slots that are filled at generation time via variable-specific sampling distributions. Continuous variables—such as signal names, timestamps, prediction horizons, and numerical thresholds—are sampled directly from the episode data or drawn from predefined distributions. Discrete variables—such as tasks, anomaly types, and root causes—are sampled from curated vocabularies. These vocabularies were initially aggregated from the open-source datasets used in FactoryBench, then substantially expanded by PhD-level robotics experts to cover a broader range of operationally realistic scenarios, while remaining fully reproducible for FactoryWave episodes. This separation between template structure and sampled content is what allows a small number of hand-authored templates to generate a large and semantically diverse question pool.

Reference to experts is a bit dangerous, if it is not clearly described how they operated. Was some protocol followed to identify the variables?

Template design. Each of the 40 question templates was manually authored to probe one specific reasoning capability at the appropriate level of the hierarchy, while remaining general enough to admit a wide range of concrete instantiations. Templates are parameterized over episode segments, signal names, event descriptions, timestamps, and predicted values. Answer options for multi-select questions are drawn from a shared pool of verifiable statements, each paired with a rule that can be evaluated deterministically against the time series, given the densely labelled data. This design ensures that ground truth is never imputed or inferred—it is computed directly from labeled episode data—making the benchmark both reliable and fully reproducible.

4.2 Density of FactoryWave

4.3 Adapting labelling to open datasets and simulations

4.4 Dataset diversity & quality assurance

A critical risk in template-generated benchmarks is a lack of semantic diversity, leading models to memorize structural patterns rather than perform true reasoning. To ensure our dataset evaluates robust machine understanding, we utilize the **Vendi Score** on the embeddings of our Q&A pairs to measure and maximize effective population diversity.

citation needed

We iteratively evaluate dataset diversity across three primary axes during generation:

- **Low Diversity (Parameter Variation):** Varying only parameters like time windows and joint indices yields a low Vendi Score, confirming that simple parameter randomization is insufficient.
- **Moderate Diversity (Format Variation):** Semantically identical questions expressed across different formats (Open-ended, Multiple Choice, True/False) measurably increase the effective diversity.
- **High Diversity (Template & Type Variation):** Introducing mixed reasoning templates across levels (e.g., kinematic comparison, derivative estimation like friction/acceleration,

Table 4: Expected performance across levels (accuracy %).

Level	Random	Current LLMs	Target (w/ prior)	Human Expert
1	50%	85–95%	98%	99%
2	20%	60–75%	85%	95%
3	10%	35–50%	70%	90%
4	5%	15–30%	45%	80%

anomaly detection) drives the highest Vendi Score. By enforcing high Vendi Scores across our generated subsets, we ensure the benchmark tests versatile analytical capabilities.

4.5 Pipeline overview

5 Experiments

5.1 Models to evaluate

Frontier LLMs:

- GPT-4o, GPT-4-Turbo
- Gemini-2.5-Pro, Gemini-2.5-Flash
- Claude-3.5-Sonnet, Claude-3-Opus

Specialized Methods:

- Time-series encoders + LLM (Chronos, Moirai)
- Multimodal industrial models (FD-LLM)
- RAG with manual retrieval

Baselines:

- Random
- Rule-based (threshold detection)
- Human expert (ceiling)

5.2 Experimental configurations

Beyond out-of-the-box evaluation, we investigate how different interventions improve performance:

- **Finetuning:** We optionally finetune open-source models on the FactoryBench Q&A dataset to establish the limit of specialized training versus general reasoning.
- **Tool-Augmented Agents:** The best-performing foundational models are granted access to external time-series prediction and anomaly detection modules, testing the synergy between LLM reasoning and domain-specific computation.

5.3 Results (expected)

Hypothesis: Semantic priors (ManualsGraph) will show largest gains at Level 4.

6 Discussion

6.1 Why this benchmark matters

For researchers: First rigorous evaluation of machine understanding across reasoning levels.

For practitioners: Quantifies which AI capabilities are ready for deployment as AI engineers.

How can people employ the benchmark? Where can it be found. Will it be maintained and expanded? Will incorrect labels be corrected? Is there a private dataset too? Are parts of the dataset designated for training and parts for testing?

For the field: Defines "machine understanding" operationally via Q&A.

6.2 Limitations

- Single machine family (by design, for depth)
- Simulation-to-real gap in synthetic episodes
- Reliance on LLMs-as-Judge for evaluation of Free-Form questions

6.3 Relation to FactoryNet

This benchmark evaluates on a **subset** of FactoryNet. The full dataset (50k+ episodes, 200k+ Q&A) will be released separately with a focus on *training* industrial world models, not just evaluation.

7 Conclusion

FactoryBench introduces a systematic approach to evaluating machine understanding via Q&A. By focusing deeply on one machine family and providing reliable answer generation at scale, we enable rigorous comparison of methods across four levels of reasoning—from basic state identification to procedure generation grounded in technical manuals.

Our physical validation protocol closes the loop from benchmark to reality, addressing the fundamental question: can AI systems that excel at industrial Q&A actually help in the real world?

References

Julien Audibert et al. Usad: Unsupervised anomaly detection on multivariate time series. *KDD*, 2020.

Raghavendra Chalapathy and Sanjay Chawla. Deep learning for anomaly detection: A survey. *arXiv preprint*, 2019.

Abhimanyu Das et al. Time series foundation models and forecasting: A survey. *arXiv preprint*, 2024.

Hoang Anh Dau et al. The ucr time series classification archive. *IEEE/CAA Journal of Automatica Sinica*, 2019.

Clive W. J. Granger. Investigating causal relations by econometric models and cross-spectral methods. *Econometrica*, 1969.

Dan Hendrycks et al. Measuring massive multitask language understanding (mmlu). *ICLR*, 2021.

Nikolay Laptev, Saeed Amizadeh, and Ian Flint. Generic and scalable framework for automated time-series anomaly detection. *KDD*, 2015.

Alexander Lavin and Subutai Ahmad. Evaluating real-time anomaly detection algorithms – the numenta anomaly benchmark. *IEEE ICMLA*, 2015.

Bryan Lim et al. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *IJF*, 2021.

Yuqi Nie et al. A time series is worth 64 words: Long-term forecasting with transformers (patchtst). *ICLR*, 2023.

Boris N. Oreshkin et al. N-beats: Neural basis expansion analysis for interpretable time series forecasting. *ICLR*, 2020.

Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.

Jonas Peters, Dominik Janzing, and Bernhard Schölkopf. *Elements of Causal Inference*. MIT Press, 2017.

Yujia Qin et al. Tool learning with foundation models. *arXiv preprint*, 2023.

Donald B. Rubin. Estimating causal effects of treatments in randomized and nonrandomized studies. *Journal of Educational Psychology*, 1974.

Lukas Ruff et al. Deep one-class classification. *ICML*, 2018.

Jakob Runge et al. Detecting and quantifying causal associations in large nonlinear time series datasets. *Science Advances*, 2019.

Timo Schick et al. Toolformer: Language models can teach themselves to use tools. *NeurIPS*, 2023.

Aarohi Srivastava et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models (big-bench). *TMLR*, 2023.

Ya Su et al. Robust anomaly detection for multivariate time series through stochastic recurrent neural network (omnianomaly). *KDD*, 2019.

Ashish Vaswani et al. Attention is all you need. *NeurIPS*, 2017.

Jason Wei et al. Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*, 2022.

Qingsong Wen et al. Transformers in time series: A survey. *IJCAI*, 2022.

Gerald Woo et al. Chronos: Learning the language of time series. *arXiv preprint*, 2024.

Haixu Wu et al. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *NeurIPS*, 2021.

Shunyu Yao et al. React: Synergizing reasoning and acting in language models. *ICLR*, 2023a.

Shunyu Yao et al. Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS*, 2023b.

Xiang Yue et al. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. *CVPR*, 2024.

Ailing Zeng et al. Are transformers effective for time series forecasting? *AAAI*, 2023.

Haoyi Zhou et al. Informer: Beyond efficient transformer for long sequence time-series forecasting. *AAAI*, 2021.

Tian Zhou et al. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. *ICML*, 2022.

A Q&A pair structure

Level 2 Example (Intervention):


```
{
  "id": "4fabb9ab-e476-48ff-812d-8fc33bb63ac3",
  "level": 2,
  "template_id": 2,
  "template_type": "intervention_outcome",
  "question": "If a continual increase of feedback_tcp_speed_5
    lasting 15 timesteps happened at the current timestep
    (T=4640ms), what would most likely happen next?
    Answer only with a 4 letter string using F and T ...",
  "options": {
    "A": "No significant effect: safety stays normal ...",
    "B": "Following the event, command and measured TCP ...",
    "C": "Following the event, the system remains ...",
    "D": "Following the event, robot current increases ..."
  },
  "answer": "FTTF",
  "provenance": {
    "dataset": "inter_aursad",
    "episode": "experiment_1",
    "subseries_start_index": 131,
    "subseries_length": 62
  },
  "context": {...}
}
```


B Benchmark inference cost

A practical concern when deploying large-scale Q&A benchmarks is the monetary cost of running inference across all questions. We derive a simple cost model and tabulate estimates for three representative frontier models under batch-API pricing.

B.1 Cost model

Let Q denote the total number of question–answer pairs in the benchmark. For each question q, the API call consumes $t_{\text{in}}^{(q)}$ input tokens (prompt, context time series, and answer options) and $t_{\text{out}}^{(q)}$ output tokens (model response). We denote the per-token prices as p_{in} and p_{out} respectively (in USD per token). The total cost is:

$$C = \sum_{q=1}^{Q} \left(p_{\text{in}} \cdot t_{\text{in}}^{(q)} + p_{\text{out}} \cdot t_{\text{out}}^{(q)} \right). \tag{1}$$

Since individual token counts vary by question, we work with corpus-level averages. Let $\bar{t}_{\text{in}}$ and $\bar{t}_{\text{out}}$ denote the mean input and output tokens per question. Then:

$$C \approx Q \cdot \left(p_{\text{in}} \cdot \bar{t}_{\text{in}} + p_{\text{out}} \cdot \bar{t}_{\text{out}} \right). \tag{2}$$

It is convenient to express prices per million tokens. Writing P_{in} and P_{out} for the price in USD per million tokens:

$$C = Q \cdot \left(\frac{P_{\text{in}} \cdot \bar{t}_{\text{in}} + P_{\text{out}} \cdot \bar{t}_{\text{out}}}{10^6} \right). \tag{3}$$

B.2 Assumptions

Input tokens. Each question payload averages approximately 30 KB, comprising the system prompt, question text, serialized time-series context window, and answer options. At an empirical ratio of 0.25 tokens per byte, this yields:

$$\bar{t}_{\text{in}} = 30{,}000 \text{ bytes} \times 0.25 \text{ tokens/byte} = 7{,}500 \text{ tokens}.$$

Output tokens. The majority of FactoryBench answer formats—Multi-Select MC (`FTTF`), Ranking (`DABC`), and Tensor (a scalar)—require only $\sim$2 output tokens. Free-form answers, which constitute at most 10% of the benchmark, produce longer responses ($\sim$300 tokens). The weighted average is therefore:

$$\bar{t}_{\text{out}} = 0.9 \times 2 + 0.1 \times 300 = 31.8 \approx 32 \text{ tokens}.$$

Batch constraints. Most batch APIs impose a maximum request file size of 50 MB. At 30 KB per question, each batch accommodates $\lfloor 50{,}000/30 \rfloor \approx 1{,}666$ questions. A full benchmark run of $Q = 200{,}000$ therefore requires $\lceil 200{,}000/1{,}666 \rceil = 120$ batches. This affects throughput scheduling but not per-token pricing.

All models are evaluated via their respective **batch processing APIs**, which offer reduced pricing with longer turnaround times (typically up to 24 hours). This is the natural choice for offline benchmark evaluation.

B.3 Model pricing

Table 5 summarizes the batch-API pricing for three frontier models as of April 2026.

Table 5: Batch-API pricing for selected frontier models (USD per million tokens, April 2026).

Model	P_{in} (\$/M tokens)	P_{out} (\$/M tokens)
Claude Opus 4.6 (Anthropic)	2.50	12.50
GPT-4o (OpenAI)	1.25	5.00
Gemini 2.5 Pro (Google)	0.625	5.00

B.4 Sample calculation for $Q = 200{,}000$

Substituting into Equation 3 with $Q = 200{,}000$, $\bar{t}_{\text{in}} = 7{,}500$, and $\bar{t}_{\text{out}} = 32$:

Claude Opus 4.6.

$$\begin{aligned} C_{\text{Opus}} &= 200{,}000 \times \frac{2.50 \times 7{,}500 + 12.50 \times 32}{10^6} \\ &= 200{,}000 \times \frac{18{,}750 + 400}{10^6} \\ &= 200{,}000 \times 0.01915 = \$\mathbf{3{,}830}. \end{aligned} \tag{4}$$

GPT-4o.

$$\begin{aligned} C_{\text{GPT-4o}} &= 200{,}000 \times \frac{1.25 \times 7{,}500 + 5.00 \times 32}{10^6} \\ &= 200{,}000 \times \frac{9{,}375 + 160}{10^6} \\ &= 200{,}000 \times 0.009535 = \$\mathbf{1{,}907}. \end{aligned} \tag{5}$$

Gemini 2.5 Pro.

$$\begin{aligned} C_{\text{Gemini}} &= 200{,}000 \times \frac{0.625 \times 7{,}500 + 5.00 \times 32}{10^6} \\ &= 200{,}000 \times \frac{4{,}687.5 + 160}{10^6} \\ &= 200{,}000 \times 0.0048475 = \$\mathbf{969.50}. \end{aligned} \tag{6}$$

B.5 Summary

Table 6: Estimated benchmark inference cost for $Q = 200{,}000$ questions (batch API).

Model	Input cost	Output cost	**Total cost**	Cost per question
Claude Opus 4.6	\$3,750.00	\$80.00	**\$3,830.00**	\$0.0192
GPT-4o	\$1,875.00	\$32.00	**\$1,907.00**	\$0.0095
Gemini 2.5 Pro	\$937.50	\$32.00	**\$969.50**	\$0.0049

These estimates show that a full benchmark run of 200k questions on a frontier model costs between roughly \$1,000 and \$4,000 using batch APIs. Costs are heavily input-dominated: output tokens contribute less than 5% of the total, since 90% of FactoryBench answers are short structured strings (2 tokens). Standard (non-batch) API pricing is typically $2\times$ higher. Note that these costs are for a *single* evaluation pass; repeated runs (e.g., for variance estimation or ablation studies) scale linearly. The per-question cost (\$0.005–\$0.02) confirms that FactoryBench is economically feasible to evaluate at scale, even on the most expensive frontier models.